

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



Hệ thống nhận diện và cảnh báo kẻ địch mạnh thời gian thực trong Genshin Impact

Hà Mạnh Dũng

Mã học phần: MAT3508
Học kỳ 1, Năm học 2025-2026

Thông tin Dự án

Học phần: MAT3508 – Nhập môn Trí tuệ Nhân tạo
Học kỳ: Học kỳ 1, Năm học 2025-2026
Trường: VNU-HUS (Trường Đại học Khoa học Tự nhiên - ĐHQGHN)
Tên dự án: Genshin Impact Enemy Alert System with YOLOv8
Ngày nộp: 06/12/2025

Thành viên nhóm

Họ tên	Mã sinh viên	Github username	Đóng góp chính
Hà Mạnh Dũng	23001845	@kusanali5002	Toàn bộ dự án

Chương 1

Lời nói đầu

Trong kỷ nguyên công nghệ 4.0, Trí tuệ nhân tạo (Artificial Intelligence - AI) và đặc biệt là lĩnh vực Thị giác máy tính (Computer Vision) đang có những bước phát triển vượt bậc, thay đổi sâu sắc cách con người tương tác với máy móc. Không chỉ giới hạn trong các bài toán công nghiệp hay y tế, AI đang dần trở thành một phần không thể thiếu trong lĩnh vực giải trí điện tử, giúp nâng cao trải nghiệm người dùng theo những cách chưa từng có.

Là một sinh viên đam mê công nghệ và thích chơi game, vì vậy sau khi suy nghĩ em đã quyết định lựa chọn một dự án mang tính cá nhân hơn một chút đó là "Xây dựng hệ thống cảnh báo kẻ địch thời gian thực trong Genshin Impact sử dụng mô hình YOLOv8".

Mục tiêu của đề tài không chỉ dừng lại ở việc tạo ra một công cụ hỗ trợ chơi game, mà quan trọng hơn là cơ hội để em vận dụng những kiến thức đã học về mạng nơ-ron tích chập (CNN), quy trình xử lý dữ liệu và triển khai mô hình AI vào một bài toán thực tế có độ phức tạp cao về môi trường và điều kiện ánh sáng.

Để hoàn thành báo cáo này, chúng em xin gửi lời cảm ơn chân thành đến các thầy trong bộ môn – người đã tận tình truyền đạt kiến thức nền tảng và định hướng tư duy cho chúng em trong suốt quá trình học Nhập môn trí tuệ nhân tạo.

Mặc dù đã rất nỗ lực hoàn thiện hệ thống với độ chính xác cao nhất có thể, song do giới hạn về thời gian, thiết bị, kinh nghiệm thực tiễn và thậm chí đã xảy ra một vài lỗi khiến dự án gần như phải làm lại từ đầu, báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ các thầy và các bạn để đề tài có thể phát triển hoàn thiện hơn trong tương lai.

Em xin chân thành cảm ơn!

Mục lục

1	Lời nói đầu	3
2	Giới thiệu	7
2.1	Tóm tắt	7
2.2	Bài toán đặt ra	7
2.3	Động lực	7
3	Phương pháp & Triển khai	9
3.1	Cơ sở lý thuyết: Object Detection	9
3.1.1	Các loại mô hình Object Detection	9
3.1.2	Về các vấn đề thực tiễn gặp phải trong Object Detection	10
3.2	Cơ sở lý thuyết: YOLOv8	10
3.2.1	YOLO	10
3.2.2	Về YOLOv8	11
3.2.3	Các phiên bản của YOLOv8	12
3.2.4	Cấu trúc của YOLOv8	12
3.2.5	Hàm mất mát (Loss) và tối ưu trong YOLOv8	14
3.2.6	Các kĩ thuật huấn luyện và tăng cường dữ liệu (augmentation)	16
3.3	Dữ liệu huấn luyện	20
3.3.1	Nguồn dữ liệu	20
3.3.2	Các đối tượng trong dự án	20
3.3.3	Công cụ xử lý dữ liệu	22
3.3.4	Chia dữ liệu vào các tập huấn luyện và kiểm thử	22
3.3.5	Kết luận sơ bộ về tập dữ liệu	22
3.4	Triển khai mô hình	22
3.4.1	Framework và biến thể của mô hình	22
3.4.2	Các biện pháp tăng cường dữ liệu	22
3.4.3	Các siêu tham số (Hyperparameters)	23
3.4.4	Tiến trình huấn luyện	23
3.4.5	Kết luận sơ bộ về kết quả huấn luyện	23
4	Thực nghiệm	25
4.1	Công cụ và thư viện được sử dụng	25
4.2	Chuẩn bị dữ liệu	25
4.3	Configuration(Cấu hình huấn luyện)	26
4.4	Kết quả huấn luyện	27
4.5	Thực quan hóa dữ liệu	27
4.6	Thiết lập cửa sổ quét màn hình và cảnh báo	29
4.6.1	Công cụ và thư viện	29
4.6.2	Cấu hình của trình cảnh báo	29
4.6.3	Thiết lập danh sách cảnh báo	29
4.6.4	Khởi tạo giọng nói	29
4.6.5	Phát giọng nói	30
4.6.6	Xử lí màn hình	30
4.6.7	Đánh giá sơ bộ về cửa sổ cảnh báo	31
5	Kết luận và hướng phát triển	33
5.1	Kết luận	33
5.2	Hướng phát triển:	33

Tài liệu tham khảo

33

Chương 2

Giới thiệu

2.1 Tóm tắt

Genshin Impact là một tựa game nhập vai hành động thể giới mở (ARPG) với đồ họa phức tạp và môi trường cũng như cảnh quan phong phú. Người chơi mới thường xuyên phải đối mặt với những quái vật nguy hiểm do mỗi mê trải nghiệm cảnh quan hoặc làm nhiệm vụ dẫn tới bị hạ gục, gây mất thời gian và phiền toái trong trải nghiệm thời gian đầu khi nhân vật chưa đạt ngưỡng sức mạnh đủ lớn.

Mục tiêu chính của dự án lần này chính là xây dựng một mô hình chạy song song với game và nhận dạng cũng như phát hiện ra những kẻ địch mạnh trong khi chơi một cách nhanh chóng và chính xác trong nhiều điều kiện thời tiết và cảnh quan khác nhau của game như đồng bằng, núi cao, bão cát, băng tuyết, ...

Trong quá trình tìm hiểu và tham khảo, YOLOv8 đã được em lựa chọn vì nó có độ chính xác cao và tốc độ xử lý ổn định trong các dự án real-time cũng như phù hợp với phần cứng thiết bị đang có. Kết quả thực nghiệm sau khi train data và chơi thử trên phần cứng GPU NVIDIA RTX 3050 cho thấy mô hình đạt độ chính xác trung bình mAP(Mean AVG Precicion) lên tới 95% với tổng cộng 19 lớp quái vật khác nhau (đã loại bỏ so với dự án cũ là 30 lớp quái vật) đồng thời duy trì tốc độ và trải nghiệm trong khi chơi game mượt mà, không bị giật lag.

2.2 Bài toán đặt ra

Nhận diện những quái vật trong Genshin Impact yêu cầu xác định chính xác vị trí và class của chúng trong rất nhiều điều kiện khác nhau. Khi trải nghiệm thực tế, người chơi đang thám hiểm di chuyển liên tục, đổi nhiều góc độ nhìn, và trong một khung hình có thể lẫn những quái vật nhỏ không phải loại nguy hiểm trong khi hành vi và đặc điểm cơ thể của chúng có những nét tương đồng vì chúng cùng một loài, tạo ra một quang cảnh vô cùng đa dạng và gây khó khăn trong việc dự đoán.

Một bài toán khác cần được nhắc đến ở đây chính là nguồn dữ liệu. Đây là một chủ đề ít được quan tâm và hầu hết các dữ liệu đã quá cũ hoặc quá ít dữ liệu, không thể đáp ứng yêu cầu đề ra của dự án.

Cơ chế anti-cheat của game cũng là một yếu tố nên được quan tâm. Mô hình phải xử lý nhanh và chính xác mà không được can thiệp vào mã nguồn của trò chơi hoặc vi phạm chính sách về phần mềm thứ ba dẫn tới tài khoản của người chơi bị cấm.

Cuối cùng thì yêu cầu được đặt ra cho dự án chính là phải tạo ra được một mô hình sử dụng YOLOv8 để nhận diện, cảnh báo nhanh, chính xác các kẻ địch mạnh trong game trong nhiều điều kiện khác nhau mà phải đảm bảo an toàn cho tài khoản của người chơi.

2.3 Động lực

Với một tựa game có cộng đồng game thủ rất lớn như Genshin Impact, việc những người chơi gặp phải những khó khăn trong khi trải nghiệm là điều không thể tránh khỏi. Trong khi số lượng những người chơi mới đang ngày càng tăng vọt, việc nghiên cứu và phát triển một công cụ như thế này giúp cho trải nghiệm của người chơi trở nên dễ dàng hơn. Tuy dự án không mang nhiều giá trị về mặt thực tiễn, nhưng với cộng đồng game thủ mà nói, đây là một công cụ có thể giúp ích cho họ.

Về mặt kỹ thuật, trong các dự án real-time, những mô hình hai giai đoạn như các phiên bản của R-CNN hoặc TridentNet thường không cho ra được kết quả mong muốn, vì vậy lựa chọn các mô hình một giai đoạn như các phiên bản của YOLO hoặc RetinaNet,... giúp giảm thiểu độ trễ xử lý trong khi vẫn giữ được độ chính xác đáng kinh ngạc. Với các phần cứng GPU phổ biến, kết hợp với các kỹ thuật augmentation dữ liệu, tối ưu siêu tham số, hệ thống có thể đạt tốc độ phản hồi nhanh và chính xác trong thời gian thực.

Chương 3

Phương pháp & Triển khai

3.1 Cơ sở lý thuyết: Object Detection

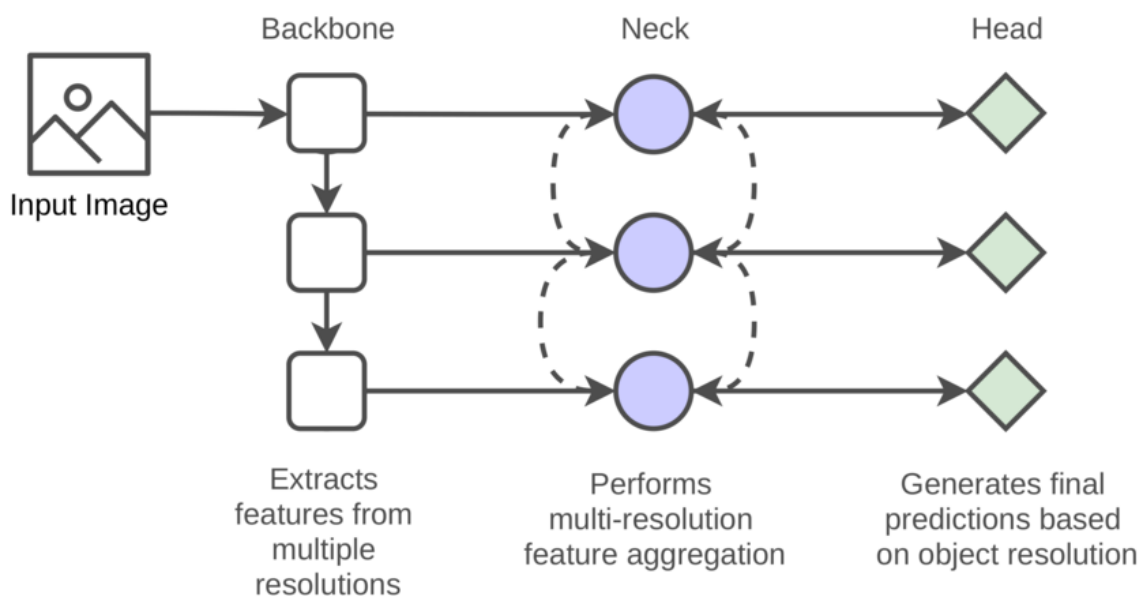
Phát hiện đối tượng (Object Detection) là một trong những bài toán lớn trong lĩnh vực Thị giác máy tính (Computer vision), góp mặt trong nhiều ứng dụng thực tế mà chúng ta đã biết như camera giám sát giao thông, nhận diện khuôn mặt, kiểm kê hàng hóa, robot, Object Detection thực hiện cả hai nhiệm vụ đó là xác định nhãn (label) của các đối tượng và xác định vị trí của chúng trong không gian ảnh thông qua các bounding box. Trong thực tế, các yếu tố như chồng lấn đối tượng, cảnh quan phức tạp, ánh sáng, màu sắc, thời tiết,... là không bất biến và thay đổi liên tục khiến cho Object Detection trở thành một bài toán vô cùng thách thức

Object Detection hiện nay có nền tảng là các mạng neuron sâu (Deep Neuron Network) và có cấu trúc gồm Backbone, Neck và Detection Head

Backbone(Feature Extraction) : có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào. Đặc trưng ở đây bao gồm cạnh, đường viền, màu sắc, và hình dạng.

Neck(Feature Fusion) : là phần trung gian, được sử dụng trong các mô hình phát hiện đối tượng hiện đại để kết hợp đặc trưng từ nhiều tầng, giúp mô hình có khả năng nhận diện đối tượng với nhiều kích thước khác nhau.

Detection Head : có nhiệm vụ dự đoán bounding box, nhãn đối tượng và điểm tự tin (confidence score). Nó thường bao gồm các lớp convolution để dự đoán tọa độ của box (x, y - cặp này thường chỉ tọa độ tâm của box và h, w tương đương chiều dài/rộng của box), nhãn lớp và độ tin cậy. Bước Non-Maximum Suppression (NMS) được sử dụng để loại bỏ các dự đoán trùng lặp.

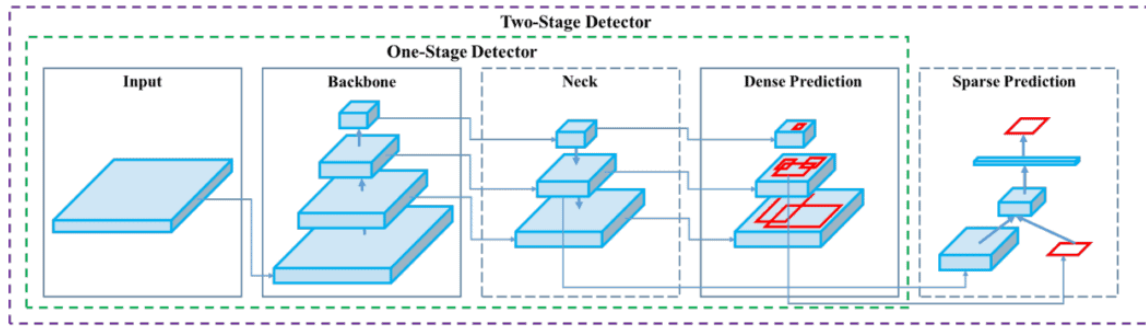


Hình 3.1: Sơ đồ mối liên hệ giữa các yếu tố backbone, neck và detection head trong Object Detection

3.1.1 Các loại mô hình Object Detection

Như đã đề cập trong phần động lực của dự án, các mô hình Object Detection được chia thành hai loại chính:

- Two-stage detectors : các mô hình đại diện gồm các phiên bản của R-CNN như Faster, Grid, Mask,... sử dụng một bước Region Proposal Network để đề xuất các vùng khả thi có thể có đối tượng. Sau đó từng vùng được phân loại và điều chỉnh bounding box. Với quy trình vận hành như thế này, các mô hình dạng two-stage detectors cho ra kết quả với độ chính xác cao, đặc biệt là với các đối tượng nhỏ và phức tạp. Tuy nhiên điểm yếu của nó là tốc độ, khiến nó bị ra rìa trong các dự án real-time vốn yêu cầu tốc độ xử lý phải rất nhanh
- One-stage detectors : các mô hình đại diện rất nổi tiếng phải kể đến là các phiên bản của YOLO, SSD,... với cách hoạt động bỏ qua bước đề xuất vùng khả thi, thay vào đó tự dự đoán bounding box và nhãn từ feature map. Với cách hoạt động này, các mô hình one-stage detectors đánh đổi một chút độ chính xác để có thể có được tốc độ xử lý nhanh, dễ triển khai trong các dự án real-time.



Hình 3.2: Tương quan giữa two-stage detectors và one-stage detectors.

3.1.2 Về các vấn đề thực tiễn gặp phải trong Object Detection

Trong thực tế, việc phát hiện vật thể gặp nhiều trở ngại như đối tượng quá nhỏ, occlusion (che khuất, chồng lấn vật thể) cũng như môi trường phức tạp.



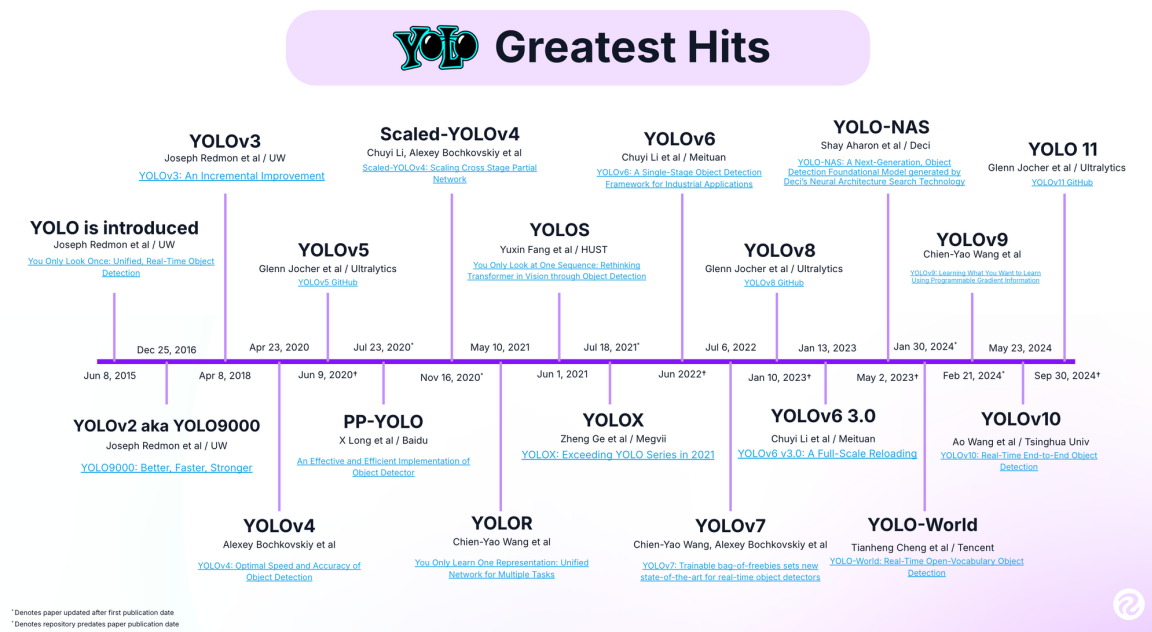
Hình 3.3: Các ảnh quái vật gắn nhãn sẵn được YOLOv8s đưa ra làm bài kiểm thử cho thấy kích thước của một vài quái rất nhỏ và được chụp trong nhiều điều kiện ánh sáng và môi trường khác nhau.

3.2 Cơ sở lý thuyết: YOLOv8

3.2.1 YOLO

YOLO (You Only Look Once) là mô hình được giới thiệu bởi tác giả gốc Joseph Redmon & Ali Farhadi vào năm 2016. YOLO vạch ra một hướng đi mới trong Object Detection bằng cách trực tiếp dự đoán bounding box

và label trong một bước duy nhất, giúp nó nhanh chóng trở thành "con cưng" trong các dự án real-time.



Hình 3.4: Lịch sử phát triển của YOLO.

Cho đến thời điểm hiện tại YOLO đã tung ra rất nhiều các biến thể khác nhau :

- **YOLOv1:** hoạt động bằng cách chia ảnh thành grid, mỗi grid dự đoán bounding box và class probability.
- **YOLOv2:** cải thiện hiệu năng và khả năng chỉnh sửa : sử dụng anchor boxes, batch normalization, multi-scale training giúp phát hiện tốt hơn nhiều hình dạng/kích thước đối tượng và có thể học nhận diện đến gần 9000 lớp nếu kết hợp classification và object detection dataset.
- **YOLOv3:** Sử dụng backbone mạnh hơn (Darknet-53), hỗ trợ multi-scale prediction → cải thiện nhận diện đối tượng nhỏ và varied size. Đây là bước tiến lớn giúp cân bằng tốc độ & độ chính xác.
- **YOLOv4:** Do nhóm khác phát triển (không phải tác giả gốc), phiên bản này dùng backbone cải tiến (CSPDarknet-53), thêm nhiều kỹ thuật mới: data augmentation (Mosaic, CutMix...), feature fusion, cấu trúc neck + head tối ưu cải thiện hiệu năng rất tốt, vẫn là sự lựa chọn tối ưu cho dự án real-time với độ chính xác cao.
- **YOLOv5-7:** YOLOv5 (từ Ultralytics) - là các phiên bản không chính thức từ nhóm gốc nhưng lại dễ sử dụng hơn : PyTorch-based, modular linh hoạt, hỗ trợ deploy thuận tiện, phù hợp cả nghiên cứu lẫn ứng dụng thực tế.
- **YOLOv8:** ra mắt năm 2023 với rất nhiều tính năng nổi bật như giảm hyperparameter, cải thiện generalization; backbone với CSP-like với module C2f; neck FPN + PAN cải tiến; hỗ trợ xuất mô hình sang các định dạng ONNX, TensorRT.
- **YOLOv9 trở đi:** tiếp tục phát triển và tích hợp nhiều cải tiến : anchor-free, tối ưu tốc độ, hỗ trợ thêm segmentation/tracking... giúp YOLO tiếp tục cạnh tranh trong bối cảnh nhiều mô hình detection hiện đại.

3.2.2 Về YOLOv8

YOLOv8 là phiên bản YOLO được phát triển và ra mắt năm 2023 bởi Ultralytics, kế thừa người đàn anh YOLOv7 của mình hướng đến cả tối ưu tốc độ lẫn độ chính xác, đồng thời hỗ trợ các nhiệm vụ detection, segmentation, pose estimation, tracking:

- **Tốc độ và hiệu suất:** YOLOv8 có khả năng hoạt động nhanh hơn và hiệu quả hơn ngay cả trên những thiết bị có tài nguyên hạn chế như điện thoại hay các thiết bị nhúng.
- **Độ chính xác:** tiếp tục cải thiện khả năng phát hiện đối tượng nhỏ và phức tạp.

- **Dễ dàng tinh chỉnh:** cung cấp khả năng dễ dàng điều chỉnh mô hình và các siêu tham số (hyperparameter) để phù hợp với các bài toán thực tế khác nhau.
- **Tích hợp tốt với các framework mới:** tận dụng tốt các framework như PyTorch và hỗ trợ việc huấn luyện cũng như triển khai qua các định dạng file như ONNX, TensorRT.

3.2.3 Các phiên bản của YOLOv8

Mỗi phiên bản của YOLO thường sẽ có nhiều biến thể để phù hợp với yêu cầu về tốc độ và hiệu năng, cho phép người dùng lựa chọn mô hình tối ưu cho phần cứng và bài toán cụ thể. YOLOv8 cũng không ngoại lệ khi tung ra 5 biến thể khác nhau:

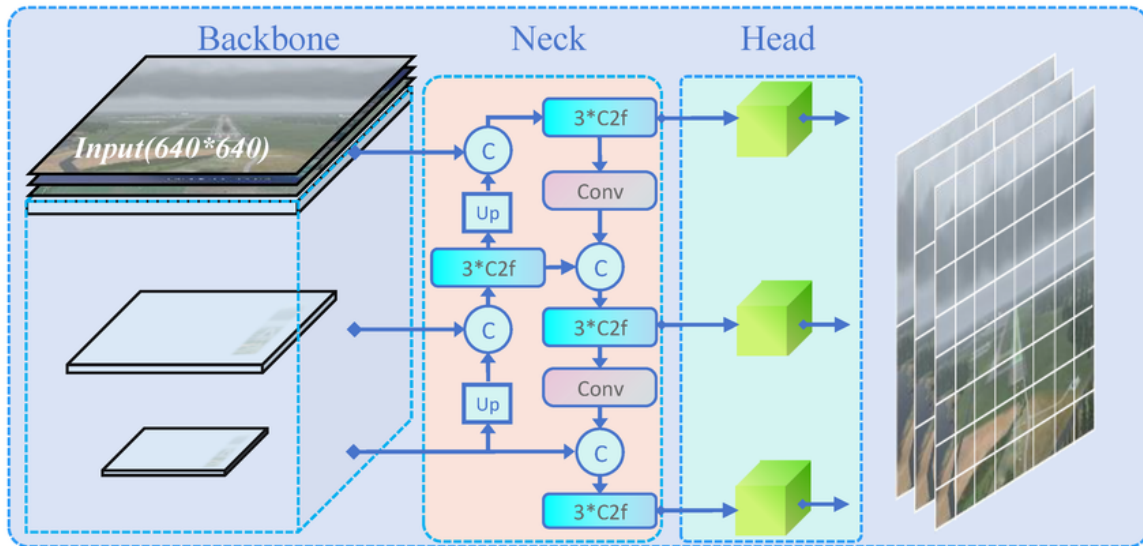
- **YOLOv8-n (Nano)** Phiên bản nhỏ nhất, tối ưu tốc độ nhất giành cho phần cứng hạn chế, đổi lại thì độ chính xác sẽ là kém nhất.
- **YOLOv8-s (Small):** cân bằng giữa tốc độ và độ chính xác, phù hợp với các ứng dụng real-time phổ biến, tuy nhiên độ chính xác vẫn chưa phải là tối ưu.
- **YOLOv8-m (Medium)** : khả năng nhận diện vật thể nhỏ khá vượt trội so với v8s
- **YOLOv8-l (Large):** phiên bản tối ưu độ chính xác nhất, rất thích hợp với dữ liệu nhiều và phức tạp, cần nhận diện chính xác nhiều đối tượng nhỏ, tuy nhiên yêu cầu về phần cứng cũng rất cao.

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Hình 3.5: So sánh hiệu năng giữa các biến thể khác nhau của YOLOv8.

3.2.4 Cấu trúc của YOLOv8

YOLOv8 có cấu trúc quen thuộc gồm ba phần: Backbone, Neck và Detection Head



Hình 3.6: cấu trúc ba phần của YOLOv8.

a. Backbone

Backbone hay còn được gọi là feature extractor (xem mục 3.1) có nhiệm vụ trích xuất đặc trưng quan trọng, chuyển raw data (dữ liệu thô) thành các feature đặc trưng mà các tầng tiếp theo có thể sử dụng được. Chất lượng của feature được trích xuất sẽ quyết định trực tiếp hiệu năng của mô hình ở các nhiệm vụ như phát hiện vật thể (detection), phân loại (classification), phân đoạn (segmentation), ước lượng pose (pose estimation).

Backbone được YOLOv8 sử dụng là **CSPDarknet53** tùy chỉnh.

Hoạt động chính

- **Trích xuất đặc trưng cấp thấp (Low-grade Feature Extraction):** ở các layer đầu, mô hình học cạnh, đường viền, góc, họa tiết đơn giản, gradient cường độ sáng. Những đặc trưng này giúp mô hình hiểu hình dạng cơ bản của vật thể.
- **Trích xuất đặc trưng cấp cao (High-grade Feature Extraction):** ở các layer sâu hơn, mô hình học hình dạng đối tượng, cấu trúc tổng thể, ngữ cảnh (context), mối quan hệ giữa các vùng.
- **Tạo đa mức độ đặc trưng (Multi-scale features):** Backbone tạo ra các feature maps với độ phân giải khác nhau. Sau đó các feature này sẽ được gửi sang Neck

b. Neck

Neck trong YOLOv8 có nhiệm vụ tổng hợp đặc trưng ở nhiều mức tỉ lệ (multi-scale feature fusion), nhằm cải thiện khả năng phát hiện đối tượng ở nhiều kích thước và điều kiện khác nhau.

Hoạt động chính

- **Hợp nhất đặc trưng đa tỉ lệ (Multi-scale Feature Fusion):** ở backbone, các feature maps được tạo ra với các kích thước khác nhau: tầng nông (80x80 đối với vật nhỏ), tầng giữa (40x40 đối với vật trung bình) và tầng sâu (20x20 đối với vật lớn). Neck kết hợp các feature này lại, đảm bảo Head về sau sẽ nhìn thấy chi tiết biên từ tầng nông và thông tin ngữ nghĩa từ tầng sâu.
- **Làm giàu đặc trưng (Feature Enhancement):** khi fusion, Neck thực hiện concat (ghép ảnh), conv 1x1 (giảm số chiều), conv 3x3 (làm mịn nhiễu), C2f block (tăng độ biểu diễn) để làm giảm thông tin nhiễu, tăng độ sắc nét cho đặc trưng, từ đó Head sẽ dễ dự đoán hơn.

Trong Neck chúng ta có đề cập đến C2f, đây chính là module được YOLOv8 sử dụng trong Neck thay vì Feature Pyramid Network (FPN). Module này kết hợp hiệu quả các đặc trưng semantic cấp cao với thông tin không gian thấp, nhờ đó nâng cao hiệu năng nhận diện với vật thể nhỏ, tối ưu hóa luồng thông tin giữa các tầng, giữ được tốc độ xử lý nhờ cấu trúc nhẹ.

c. Head

Head là phần cuối của mô hình YOLO, có nhiệm vụ chuyển các feature maps đã được trích xuất và hợp nhất (từ Backbone + Neck) thành các dự đoán cuối cùng, bao gồm tọa độ bounding box (x,y,h,w), độ tin cậy của đối tượng (object score) và xác suất của từng lớp (class probability).

Hoạt động chính

- **Phát hiện đa tỉ lệ (Multi-scale Prediction):** nó sẽ sử dụng nhiều feature với 3 scale đã được nêu ở Neck. Mỗi scale sẽ có một head dự đoán riêng. Điều này giúp YOLO phát hiện tốt vật nhỏ lẫn vật lớn và giữ được tốc độ xử lý nhanh chóng.

- **Dự đoán bounding box** : Head sẽ dự đoán bốn giá trị hình học là x,y (cặp giá trị chỉ tọa độ tâm của box), w (chiều rộng của box) và h(chiều cao của box).
- **Đánh giá độ tin cậy (Confidence Scoring)**:
 - Mỗi bounding box sẽ được head dự đoán độ tin cậy đối tượng (objectness score) : giá trị càng tiến đến 1 thì tỉ lệ có vật thể càng cao và nếu càng tiến đến 0 thì khả năng cao nó chỉ là background.

$$obj = \sigma(p_{obj}) \quad (3.1)$$

- Head cũng dự đoán class probability sau đó kết hợp với objectness score để đưa ra đánh giá cuối cùng:

$$final\ score = confidence\ score \times classprobability \quad (3.2)$$

Về cấu trúc chi tiết, YOLOv8 sử dụng decoupled head (head phân 2 nhánh) gồm regression dự đoán bounding box và objectness score; classification dự đoán class probability.

3.2.5 Hàm mất mát (Loss) và tối ưu trong YOLOv8

YOLOv8 sử dụng ba thành phần chính của Loss :

- Bounding box regression loss (loss hồi quy hộp giới hạn)
- Objectness loss (Confidence loss)
- Classification loss (Loss phân loại)

Điểm đặc biệt của YOLOv8 so với các đàn anh của nó là:

- Dùng CIoU/SIoU/DFL (Distribution Focal Loss) cho độ chính xác cao hơn.
- Dùng Binary Cross Entropy (BCE) cho objectness và class

Công thức tổng mất mát:

$$L = \lambda_{box}L_{box} + \lambda_{obj}L_{obj} + \lambda_{cls}L_{cls} \quad (3.3)$$

trong đó λ_{box} , λ_{obj} , λ_{cls} là các hệ số điều chỉnh (thường gặp nhất là 7.5, 1.0, 0.5) để cân bằng giữa độ chính xác định vị và phân lớp.

• Box Regression Loss

YOLOv8 sử dụng CIoU cải tiến từ IoU, GIoU, DIoU để tối ưu vị trí, kích thước và độ trùng khớp của bounding box:

$$L_{CIoU} = 1 - IoU + \frac{\rho^2(b, b_{gt})}{c^2} + \alpha v \quad (3.4)$$

trong đó :

- **IoU** : Đo mức độ chồng lấp giữa hộp dự đoán và hộp thực.

$$IoU = \frac{|B \cap B_{gt}|}{|B \cup B_{gt}|} \quad (3.5)$$

- $\rho^2(b, b_{gt})$ là khoảng cách bình phương giữa tâm hai hộp.

$$\rho^2(b, b_{gt}) = (x - x_{gt})^2 + (y - y_{gt})^2 \quad (3.6)$$

- **c** là đường chéo bao phủ hai box (enclosing box).

$$c^2 = (w + w_{gt})^2 + (h + h_{gt})^2 \quad (3.7)$$

- **v** là thành phần phạt độ lệch tỉ lệ chiều rộng/chiều cao.

$$v = \frac{4}{\pi^2} \left(\arctan \frac{w_{gt}}{h_{gt}} - \arctan \frac{w}{h} \right)^2 \quad (3.8)$$

- α là hệ số cân bằng phụ thuộc IoU.

$$\alpha = \frac{v}{(1 - \text{IoU}) + v} \quad (3.9)$$

Có ba phần trong công thức CIOU Loss mà ta cần chú ý:

- * **1 - IoU** : thành phần phạt thiếu chồng lấp.
- * $\frac{p^2}{c^2}$: thành phần phạt lệch tâm
- * αv : thành phần phạt tỷ lệ hộp.

- **Distribution Focal Loss (DFL) :**

YOLOv8 dùng DFL để học chính xác vị trí biên box (left, right, bottom, top). Tuy nhiên nó không học và dự đoán trực tiếp tọa độ box mà dự đoán phân phối rời rạc trên các bins.

$$L_{\text{DFL}} = - \sum_i p_i \log q_i \quad (3.10)$$

trong đó p_i là phân phối mục tiêu (one-hot hoặc soft), q_i là phân phối mô phỏng (predict distribution của bounding box).

- **Classification Loss :**

Classification Loss là thành phần hàm mất mát dùng để đo mức độ chính xác khi mô hình dự đoán loại đối tượng (class label) cho mỗi bounding box.

YOLOv8 sử dụng Binary Cross-Entropy Loss with Logits (BCEWithLogitsLoss) cho phân loại đa lớp (multi-label classification). Đối với xử lý từng lớp, YOLOv8 sử dụng BCE theo công thức :

$$L_{\text{cls}} = - \sum_{k=1}^C [y_k \log(\sigma(p_k)) + (1 - y_k) \log(1 - \sigma(p_k))] \quad (3.11)$$

trong đó:

- C : số lớp.
- p_k : raw logit mô hình dự đoán cho lớp k.
- $\sigma(p_k)$: sigmoid - xác suất thuộc lớp k.
- $y_k \in \{0, 1\}$: nhãn thật.
 - * **1** : là lớp đúng.
 - * **0** : không phải lớp đúng.

Softmax bị loại bỏ mà thay vào đó là BCE bởi vì YOLOv8 chuyên về xử lý multi-label (đa lớp) nơi mà một nhãn có thể chứa hai hay nhiều lớp cùng lúc, trong khi đó Softmax luôn giới hạn tổng các xác suất phải bằng 1, không thích hợp cho multi-label.

Ngoài ra, trong YOLOv8 mở rộng, Classification Loss còn được cộng thêm hệ số Focal để giảm ảnh hưởng mẫu dễ.

$$L_{\text{cls}} = - \sum_{k=1}^C [\alpha y_k (1 - \sigma(p_k))^\gamma \log(\sigma(p_k)) + (1 - \alpha)(1 - y_k) (\sigma(p_k))^\gamma \log(1 - \sigma(p_k))] \quad (3.12)$$

trong đó

- α là cân bằng lớp.
- γ là điều khiển phạt mẫu dễ (thường có giá trị bằng 2).

3.2.6 Các kĩ thuật huấn luyện và tăng cường dữ liệu (augmentation)

Các kĩ thuật huấn luyện và tăng cường dữ liệu giúp YOLOv8 tăng khả năng generalization, giảm overfitting và tăng hiệu suất phát hiện đối tượng trong các điều kiện thực tế khác nhau.

Các phương pháp augmentation thực chất là các phép chỉnh sửa màu sắc, hình dạng hoặc kết hợp ảnh mà chúng ta đã quen thuộc trong các trình chỉnh sửa ảnh hoặc video như Photoshop, After Effect hay Davinci Resolve :

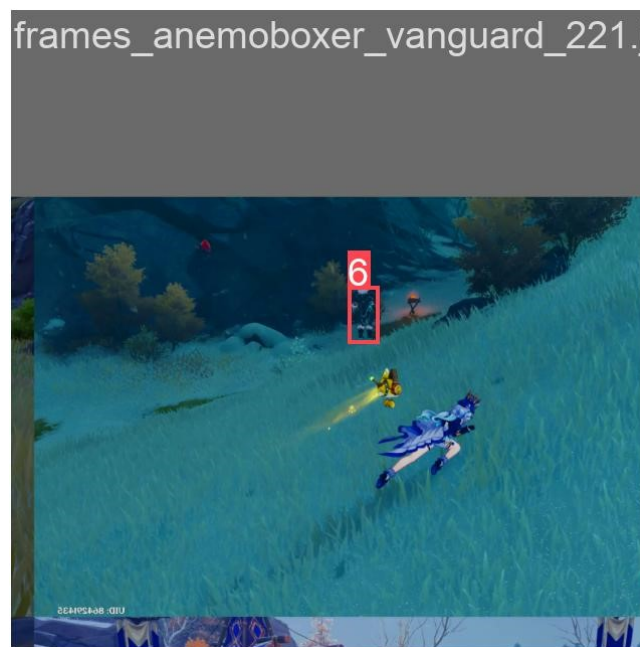
- Mosaic (Kết hợp ảnh).
- MixUp (Trộn ảnh).
- Color Augmentation (HSV Augment).
- Random Flip Vertical/Horizontal (Lật ảnh ngang/dọc ngẫu nhiên).
- Random Scaling /Cropping (Tăng/giảm độ lớn / Cắt ngẫu nhiên).
- Blur, Noise, Cutout

a. Mosaic

Mosaic lần đầu xuất hiện trong YOLOv4 và vẫn tiếp tục được sử dụng trong các phiên bản YOLO kế tiếp. Mosaic thực hiện ghép 4 ảnh khác nhau thành một ảnh lớn duy nhất theo tỉ lệ và vị trí khác nhau, giúp mô hình học được các mối quan hệ ngữ cảnh giữa các đối tượng trong nhiều điều kiện khác nhau.

Cách hoạt động

- Chọn ngẫu nhiên 4 ảnh từ dataset.
- Chia mỗi ảnh thành 1/4 của ảnh kết quả.
- Ghép các ảnh đã được chọn và chia.
- Chỉnh lại bounding box sao cho khớp với vị trí mới của các đối tượng.



Hình 3.7: Mosaic thực tế trong dự án.

Ưu điểm

- **Tăng tính đa dạng cảnh quan** : mô hình xác định được đối tượng trong nhiều cảnh quan khác nhau.
- **Tăng tính thích nghi đối với vật thể nhỏ** : việc ghép ảnh vào như vậy khiến cho vật thể nhỏ chiếm nhiều diện tích ảnh hơn so với ảnh ban đầu, giúp mô hình có thể dễ dàng thấy nó hơn.

- **Tiết kiệm bộ nhớ** : Mosaic làm giàu dữ liệu mà không cần chiếm dụng quá nhiều bộ nhớ

Nhược điểm

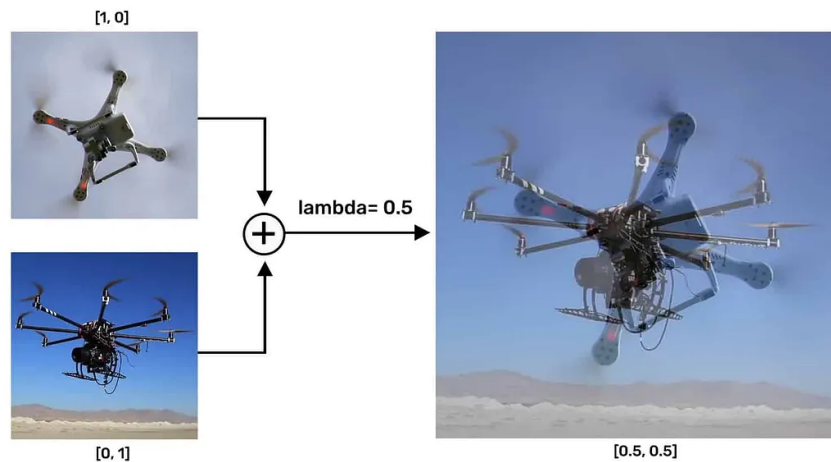
- **Không giống ngữ cảnh thực tế trong Genshin** : việc ghép có thể tạo ra ngữ cảnh kém tự nhiên so với thực tế trò chơi, dẫn đến mô hình học các quan hệ không có thật giữa quái vật và địa hình (ví dụ như mô hình có thể hiểu lầm rằng Chó săn ma vật có thể xuất hiện ở Liyue).
- **Khó cân bằng nhãn** : một số lớp có thể xuất hiện ít hơn hoặc nhiều hơn trong một ảnh được mosaic, dẫn tới mất cân bằng tạm thời trong batch.

b. MixUp

MixUp là loại kĩ thuật được sử dụng để cải thiện khả năng tổng quát hóa của mô hình. Nó cho phép mixup (trộn) hai ảnh và nhãn tương ứng để tạo ra một ảnh mới.

Cách hoạt động

- Chọn ngẫu nhiên hai ảnh từ dataset.
- Chọn hệ số trộn λ ngẫu nhiên trong khoảng $\{0,1\}$.
- Tạo ảnh mới : $I = \lambda I_1 + (1 - \lambda)I_2$, trong đó I là ảnh mới, I_1 và I_2 lần lượt là các ảnh đã được chọn trong dataset.
- Thực hiện tạo combination label đối với các nhãn : $\lambda_{\text{mix}} = \lambda \text{label}_1 + (1 - \lambda) \text{label}_2$
- Điều chỉnh bounding box nếu cần.



Hình 3.8: Mixup.

Ưu điểm

- **Giảm overfitting** : tránh việc mô hình học quá kĩ dữ liệu huấn luyện dẫn tới ghi nhớ dữ liệu cụ thể thay vì học dữ liệu tổng quát.
- **Smooth label distribution** : làm nhãn "mềm" hơn, tránh khiến mô hình bị overconfident, bị ép phải tin và đưa ra kết quả 100% vào một lớp vì ảnh đã được hòa trộn.

Nhược điểm

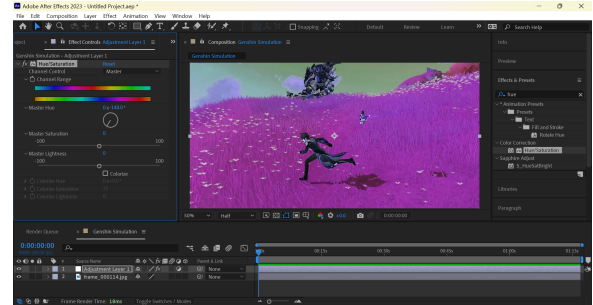
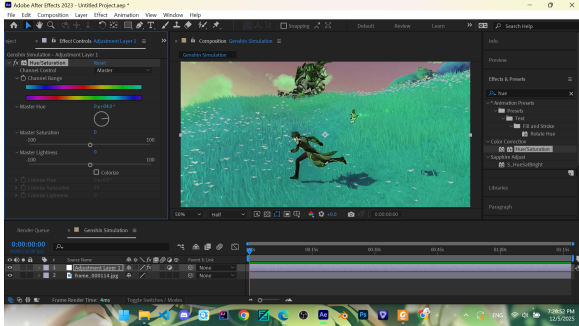
- Khiến ảnh kém tự nhiên, trong detection hay vô tình làm nhòe object, tạo màu sắc lạ, gây khó học localization chính xác.
- **Không phù hợp với tất cả các loại bài toán** : trong các bài toán tương tự như dự án này vì sẽ phải căn chỉnh bounding box / mask vô cùng cẩn thận

Trong dự án này, việc sử dụng Mixup nên chỉ ở mức rất thấp.

c. Color Augmentation (HSV Augment).

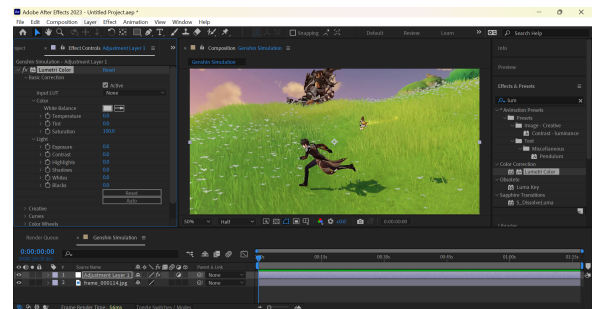
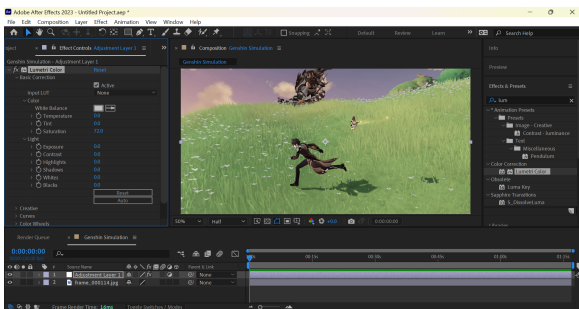
Đây là kỹ thuật khá phổ biến trong chỉnh sửa ảnh và video bao gồm Hue (Dịch tông màu), Saturation (Độ bão hòa màu) và Value/Bright (Độ sáng) và một vài tính chỉnh mở rộng như Contrast (tương phản), dùng để mô phỏng các điều kiện ánh sáng, màu sắc và chất lượng ảnh khác nhau từ dữ liệu gốc trong thực tế, trong đó :

• Hue



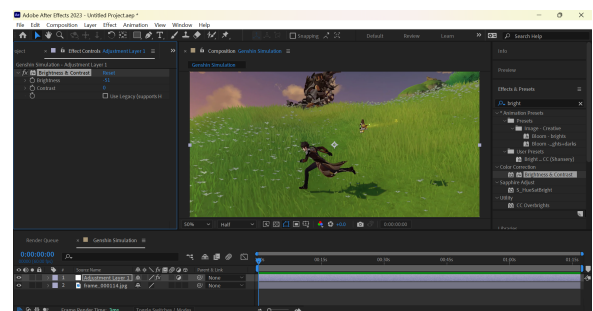
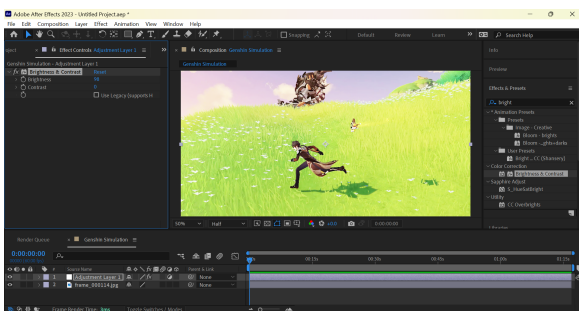
Hình 3.9: Mô phỏng ảnh hưởng của Hue đến tông màu với ảnh lấy từ dataset bằng After Effect

• Saturation



Hình 3.10: Mô phỏng ảnh hưởng của Saturation đến độ bão hòa màu với ảnh lấy từ dataset bằng After Effect

• Brightness/Value



Hình 3.11: Mô phỏng ảnh hưởng của Brightness/Value đến độ sáng với ảnh lấy từ dataset bằng After Effect

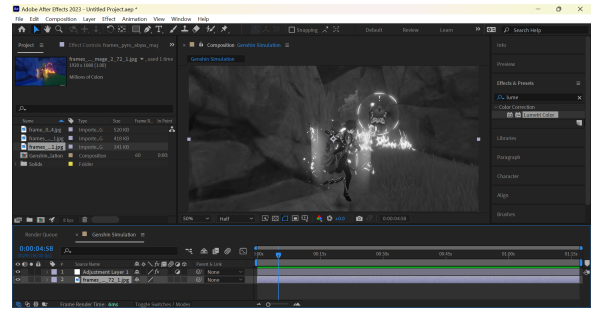
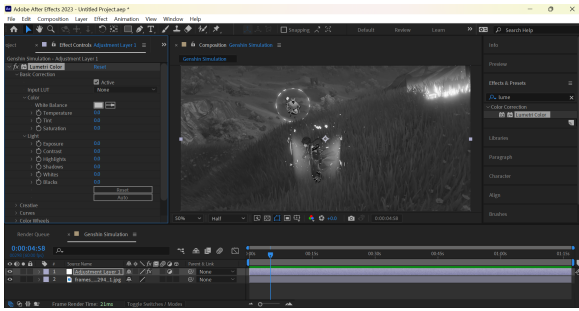
Ưu điểm

- Giúp mô hình phát hiện đối tượng ổn định trong nhiều điều kiện ánh sáng, thời tiết, góc độ khác nhau, thích ứng trên nhiều dòng máy.
- Đặc biệt phát huy tác dụng với các dự án nhận diện vật thể trong khi chuyển động liên tục.

Nhược điểm

- Điều chỉnh các thông số một cách cực đoan có thể làm mất các đặc trưng quan trọng của vật thể.

- Gây tác dụng tiêu cực đến các bài toán phân loại chỉ dựa vào màu sắc.
- Thay đổi các chỉ số làm sai lệch môi trường nhận diện so với thực tế.



Hình 3.12: Pháp sư vực sâu Băng (trái) và Pháp sư vực sâu Hỏa (phải) rất khó để phân biệt khi chịu tác động mạnh của Saturation.

d. Random Flip Vertical/Horizontal

Đây là một phương pháp khá đơn giản mà hiệu quả trong tăng cường dữ liệu, khi chỉ cần lật ngang hoặc dọc theo hệ trục tọa độ.

Ưu điểm

- Giúp mô hình nhận diện đối tượng theo nhiều hướng.
- Là công cụ sinh dữ liệu ít lỗi mà vô cùng an toàn nếu dataset ít dữ liệu hoặc các vật thể xuất hiện theo hướng không đồng nhất

Nhược điểm

- Đôi khi tạo ra dữ liệu không cần thiết và phi thực tế (ví dụ như chẳng có quái vật nào lộn ngược trong Genshin Impact cả vì đây là trò chơi theo góc nhìn thứ ba).

Trong dự án này vertical flip không được khuyến khích sử dụng.

e. Blur/Noise, Cutout

Blur/Noise là kỹ thuật cố tình làm mờ hoặc gây nhiễu ảnh để mô phỏng chất lượng hình ảnh tệ do di chuyển quá nhanh hoặc đơn giản là thiết bị ghi hình kém chất lượng.

Ưu điểm

- Mô phỏng tốt các tình huống nhòe hình do di chuyển nhanh, đặc biệt là trong game.



Hình 3.13: Blur thực tế ở các mức độ khác nhau trong dự án.

Tổng hợp về mô hình đã chọn : YOLOv8

Dự án này có các yêu cầu khá quan trọng như sau :

- Phát hiện kẻ địch theo thời gian thực : người chơi liên tục di chuyển đòi hỏi mô hình phải có độ nhanh nhạy và chính xác cao.

- Hoạt động ổn định trên các phần cứng tầm trung như GTX 1650, 1660, RTX 2060,... hoặc CPU khá trong quá trình chơi game, training với thời gian không quá lâu và không bị crash hoặc hư hỏng thiết bị trên RTX 3050.
- nhận dạng tốt trên môi trường 3D, ánh sáng thay đổi, nhiều hiệu ứng màu sắc dày đặc.
- Triển khai dễ dàng, tích hợp vào cửa sổ đọc màn hình
- Tối ưu tốc độ inference cao (>40 FPS)

Từ những yêu cầu trên, YOLOv8 là một trong những lựa chọn mạnh nhất trên các mặt tốc độ, độ chính xác lẫn tính đơn giản trong triển khai.

Tuy nhiên nó vẫn có một vài nhược điểm nhỏ

- Như đã phân tích trước đó, phần cứng đề xuất tối thiểu để có thể huấn luyện là RTX 3050 hoặc tương đương với YOLOv8s, vì vậy đối với các biến thể như v8m hoặc v8l, hoặc nếu sử dụng v8s để huấn luyện với phần cứng thấp hơn có thể sẽ tốn quá nhiều thời gian, crash hoặc gây hư hỏng thiết bị. Vì vậy YOLOv8 yêu cầu khá cao về phần cứng.
- Trong điều kiện vật thể cần phát hiện quá nhỏ hoặc xảy ra chồng lấn mạnh, mô hình cũng có hiệu năng kém đi rõ rệt. Tuy nhiên các quái vật lớn trong Genshin Impact thường không xuất hiện với số đông nên sẽ không ảnh hưởng quá nhiều đến dự án.
- Đôi khi anchor-box tạo bounding box hơi hẹp hoặc hơi rộng ở các vật thể nhỏ. Tuy nhiên điều này không ảnh hưởng nhiều đến dự án.
- Dữ liệu cần gán nhãn thật chính xác. Do dataset của dự án này được làm hoàn toàn bằng tay, độ chính xác của nhãn có thể không được như ý, dẫn đến nhầm lẫn theo chuỗi trong YOLOv8 khi nhận diện vật thể.
- YOLOv8 cũng phụ thuộc rất mạnh vào các kỹ thuật tăng cường dữ liệu. Vậy nên nếu dataset có quá ít dữ liệu hoặc quá đơn giản thì mô hình sẽ dễ bị overfitting.

3.3 Dữ liệu huấn luyện

3.3.1 Nguồn dữ liệu

Theo tóm tắt đề tài ban đầu, tập dữ liệu (dataset) sẽ được lấy từ Roboflow, tuy nhiên do dữ liệu quá ít và đã cũ nên phương án này đã bị loại bỏ.

Thay vào đó chúng ta sẽ tự tạo ra dataset bằng cách trực tiếp vào game, đi tìm những nơi có quái vật nguy hiểm trú ngụ, đi xung quanh và ghi màn hình trong khoảng từ 1-2 phút. Sau đó chúng ta sẽ sử dụng thư viện os và cv2 của Python để tạo một chương trình trích xuất các frame và lưu thành ảnh. Làm tương tự như vậy với nhiều vị trí khác nhau mà quái vật trú ngụ để có thể thu được dữ liệu phong phú cho nhiều góc độ và môi trường.

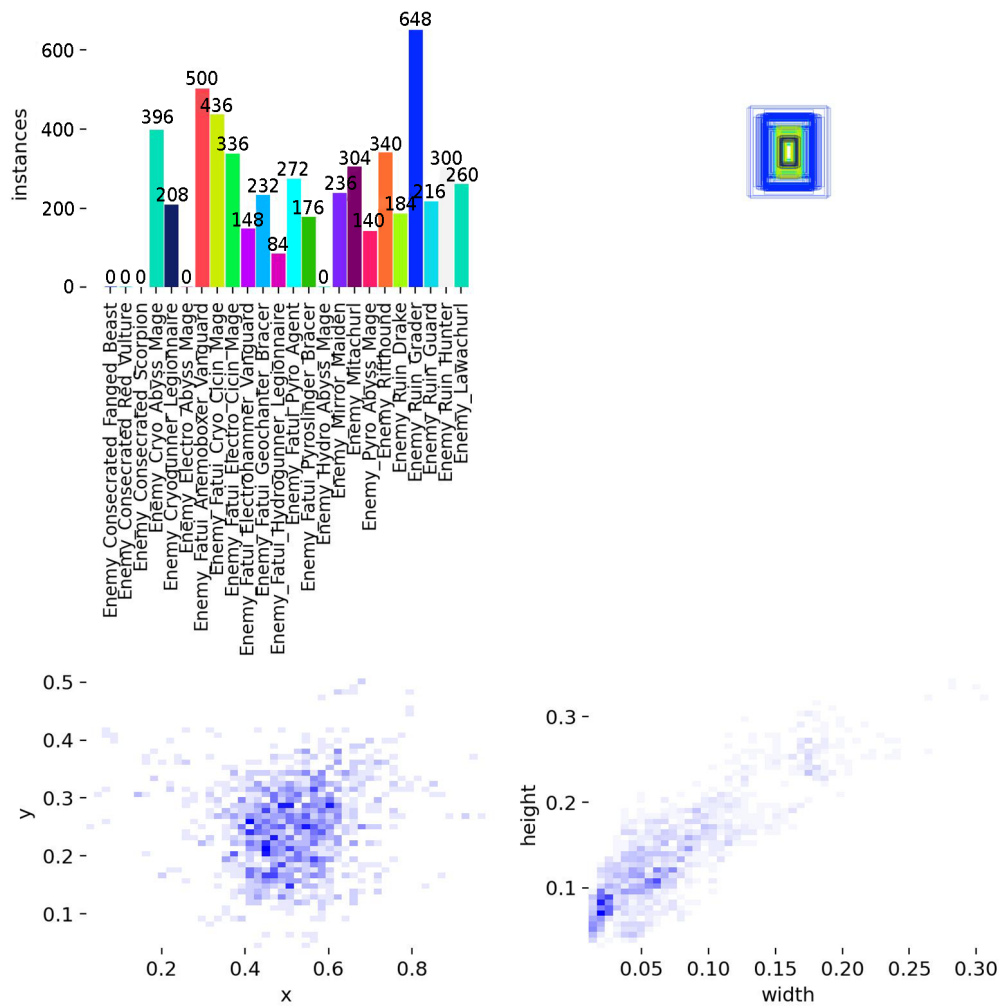
Với mỗi video có độ dài từ 1-2 phút, chúng ta sẽ có khoảng từ 720-1440 ảnh, mỗi quái vật thường sẽ được ghi lại bởi khoảng 3-4 video, và với tổng cộng 19 loại quái vật, chúng ta sẽ có dữ liệu thô gồm khoảng 41000-109000 ảnh.

Sau khi có được các ảnh đã trích xuất, chúng ta sẽ đi qua từng ảnh và xem xét chúng có đủ điều kiện để làm ảnh mẫu hay không. Mặc dù cách làm này vô cùng tốn thời gian, tuy nhiên quá trình này là phải chấp nhận bởi vì nếu sử dụng các công cụ auto-label thì rất có thể nó sẽ bounding box sai cho đối tượng hoặc bỏ sót đối tượng. Sau quá trình này tập dữ liệu gốc còn lại khoảng gần 7000 ảnh.

3.3.2 Các đối tượng trong dự án

Sau khi cân nhắc các loại quái vật phù hợp với mục tiêu của dự án, sẽ có tổng cộng 19 đối tượng khác nhau thuộc nhiều chủng loài :

- **Cryo Abyss Mage** : Pháp sư Vực sâu Băng.
- **Pyro Abyss Mage** : Pháp sư Vực sâu Hỏa.

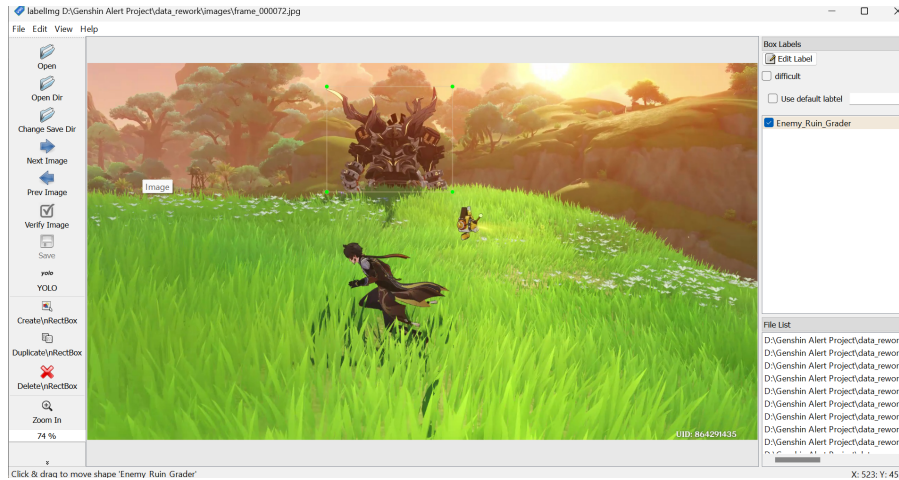


Hình 3.14: Tổng quan về dữ liệu được sử dụng.

- **Fatui Skirmisher - Cryogunner Legionnaire** : Tiên phong Fatui - Vệ binh Băng hạng nặng.
- **Fatui Skirmisher - Hydrogunner Legionnaire** : Tiên phong Fatui - Vệ binh Thủy hạng nặng.
- **Fatui Skirmisher - Anemoboxer Vanguard** : Tiên phong Fatui - Tay đấm Phong.
- **Fatui Electro Cicin Mage** : Thuật sĩ Cicin Lôi.
- **Fatui Cryo Cicin Mage** : Thuật sĩ Cicin Băng.
- **Fatui Skirmisher - Pyroslinger Bracer** : Tiên phong Fatui - Quân du kích Hỏa.
- **Fatui Skirmisher - Geochanter Bracer** : Tiên phong Fatui - Quân du kích Nham.
- **Fatui Pyro Agent** : Fatui - Người xử lý nợ.
- **Mirror Maiden** : Thiếu nữ kính.
- **Mitachurl** : Mitachurl.
- **Rifthound** : Chó săn ma vật.
- **Ruin Drake** : Rồng Di tích.
- **Ruin Guard** : Thủ vệ Di tích.
- **Ruin Grader** : Cần cẩu Di tích.
- **Ruin Hunter** : Thợ săn Di tích.
- **Lawachurl** : Lawachurl.

3.3.3 Công cụ xử lý dữ liệu

Dự án lựa chọn LabelImg để dán nhãn thủ công cho các ảnh.



Hình 3.15: Giao diện của LabelImg

LabelImg cho phép người dùng vẽ bounding box xung quanh vật thể cần gán nhãn trong ảnh và lưu nhãn dưới định dạng YOLO đọc được đó là .txt.

3.3.4 Chia dữ liệu vào các tập huấn luyện và kiểm thử

Tiến hành chia tổng số 6576 ảnh và file label tương ứng vào 2 thư mục :

- **Train (5261 ảnh chiếm 80%)** : dữ liệu dùng để huấn luyện mô hình, đảm bảo nó học và nhận dạng được các đối tượng mục tiêu trong các điều kiện khác nhau mà mình đã thu thập.
- **Validation (1315 ảnh chiếm 20%)** : ngăn tình trạng mô hình "học vẹt" vì đã biết tất cả dữ liệu huấn luyện, giúp mô hình thống kê các chỉ số để biết mô hình đang học đúng hay sai từ đó đưa ra chiến thuật phù hợp, từ đó mô hình sẽ chọn ra được vòng lặp có kết quả tốt nhất dùng để chạy thực tế.

3.3.5 Kết luận sơ bộ về tập dữ liệu

Tập dữ liệu tự thu thập và tự tạo này cung cấp cơ sở dữ liệu khá phong phú về các kẻ địch mạnh trong Genshin Impact, đủ đáp ứng yêu cầu cho YOLOv8 có thể học và nhận diện trong khi trải nghiệm thực tế.

3.4 Triển khai mô hình

3.4.1 Framework và biến thể của mô hình

Với phần cứng GPU là NVIDIA RTX 3050, cộng thêm yêu cầu của dự án, sau khi cân nhắc, dự án sẽ sử dụng framework Ultralytics YOLOv8 phiên bản Small (yolov8s). Trọng số pretrained trên dataset COCO cũng sẽ được sử dụng giúp rút ngắn thời gian huấn luyện so với huấn luyện từ đầu.

3.4.2 Các biện pháp tăng cường dữ liệu

Dự án sử dụng các biện pháp tăng cường dữ liệu sau :

- Mosaic
- HSV
- Random Erasing
- MixUp

3.4.3 Các siêu tham số (Hyperparameters)

Các siêu tham số được cân nhắc kĩ lưỡng để cân bằng giữa sức mạnh phần cứng, kết quả kì vọng và thử nghiệm thực tế :

- Epochs : là số vòng lặp mô hình sẽ đi qua toàn bộ dữ liệu cần được huấn luyện. Đây là con số phù hợp với lượng dữ liệu nhỏ cận trung bình của dự án.
- Batch size : 8 ảnh
- Learning rate : $lr0 = 0.001$
- Patience : patience = 15 phòng trường hợp không dừng tiến trình huấn luyện khi các epoch kế tiếp không đạt tăng tiến về hiệu năng.

3.4.4 Tiến trình huấn luyện

Tiến trình huấn luyện của dự án bao gồm :

- **Load dataset** : tiến hành cho các dữ liệu train và val đã được chuẩn hóa cho YOLO.
- **Tăng cường dữ liệu** : thêm các kỹ thuật augmentation như mosaic, mixup, flip (horizontal), HSV coloring và random erasing.
- **Khởi chạy huấn luyện** : bắt đầu huấn luyện setting đã có.
- **Evaluate**: đánh giá hiệu năng của mô hình trên các ảnh valpredict mà mô hình cung cấp cũng như biểu đồ, điều chỉnh và train lại với dữ liệu cũ nếu muốn.
- **Thu kết quả** : tìm file best.pt (là kết quả tốt nhất mô hình có thể tạo ra trong quá trình huấn luyện) trong thư mục kết quả huấn luyện và sử dụng nó cho các bước tiếp theo của dự án. Có thể chuyển sang các định dạng như ONNX/TensorRT nếu cần.

3.4.5 Kết luận sơ bộ về kết quả huấn luyện

Quy trình triển khai huấn luyện bằng YOLOv8 trong dự án :

- Mô hình được học những dữ liệu thực tế từ trong game dưới nhiều cảnh quan, góc độ, thời tiết khác nhau
- Các siêu tham số và kỹ thuật tăng cường được tối ưu .
- Mô hình trả về file trọng số tốt nhất, sẵn sàng cho dự án cảnh báo kẻ địch mạnh trong Genshin.

Chương 4

Thực nghiệm

4.1 Công cụ và thư viện được sử dụng

Trong quá trình thử nghiệm trình cảnh báo, những công cụ sau đã được sử dụng trong dự án :

- **Python 3.12.8.**
- **Ultralytics YOLOv8** - biến thể được sử dụng là **yolov8s** (Small).
- **PyTorch 2.7.1 + CUDA 11.8.**
- **OpenCV 4.12.0.88, Pillow**
- **NumPy**
- **pyttsx3** : dùng để phát âm thanh cảnh báo bằng giọng nói.

4.2 Chuẩn bị dữ liệu

Dữ liệu sẽ được sắp xếp và tổ chức theo chuẩn YOLO :

```
images/  
  train/  
  val/  
labels/  
  train/  
  val/
```

Theo đó, file data.yaml sẽ chịu trách nhiệm lưu đường dẫn và mô tả dữ liệu theo như cây phía trên, kèm vào đó là lưu các class với ID tương ứng :

```
path: .  
train: images/train  
val: images/val  
  
name:  
  0: Enemy_Consecrated_Fanged_Beast  
  1: Enemy_Consecrated_Red_Vulture  
  2: Enemy_Consecrated_Scorpion  
  ...  
  21: Enemy_Ruin_Guard  
  22: Enemy_Ruin_Hunter  
  23: Enemy_Lawachurl
```

Do sơ suất, dự án đã từng bị mất hết dữ liệu một lần và khi tạo lại dữ liệu mới đã quên xóa những class không tồn tại trong dự án, tuy nhiên điều này không ảnh hưởng quá nhiều đến kết quả huấn luyện khi ID tương ứng của các class vẫn được hiển thị đúng trong label. Đó chính là lí do mặc dù dự án thực tế chỉ có 19 đối tượng nhưng lại có đến 24 class trong file data.yaml.

4.3 Configuration(Cấu hình huấn luyện)

Các thông số huấn luyện được thiết lập như sau :

- Model : yolov8s
- Batch size : 8
- Image size : 960x960
- Epoch : 100
- Optimizer : AdamW
 - lr0 = 0.001
 - weight decay = 0.0005
- Augmentation :
 - hsv_h = 0.015
 - hsv_s = 0.7
 - hsv_v = 0.4
 - scale = 0.5
 - fliplr(flip horizontal) = 0.5
 - mosaic = 0.7
 - mixup = 0.1
- Early stop : patience = 15

Mã nguồn sử dụng để khởi chạy huấn luyện :

```
def train():
    print("Using CUDA:", torch.cuda.is_available())
    model = YOLO("yolov8s.pt")

    results = model.train(
        data="data.yaml",
        imgsz=960,
        epochs=100,
        batch=8,
        workers=4,
        device=0,
        patience=15,

        #augmentation
        hsv_h=0.015,
        hsv_s=0.7,
        hsv_v=0.4,
        degrees=0.0,
        translate=0.1,
        scale=0.5,
        shear=0.0,
        flipud=0.0,
        fliplr=0.5,
        mosaic=0.7,
        mixup=0.1,

        lr0=0.001,
        lrf=0.01,
        weight_decay=0.0005,
        warmup_epochs=3,
```

```

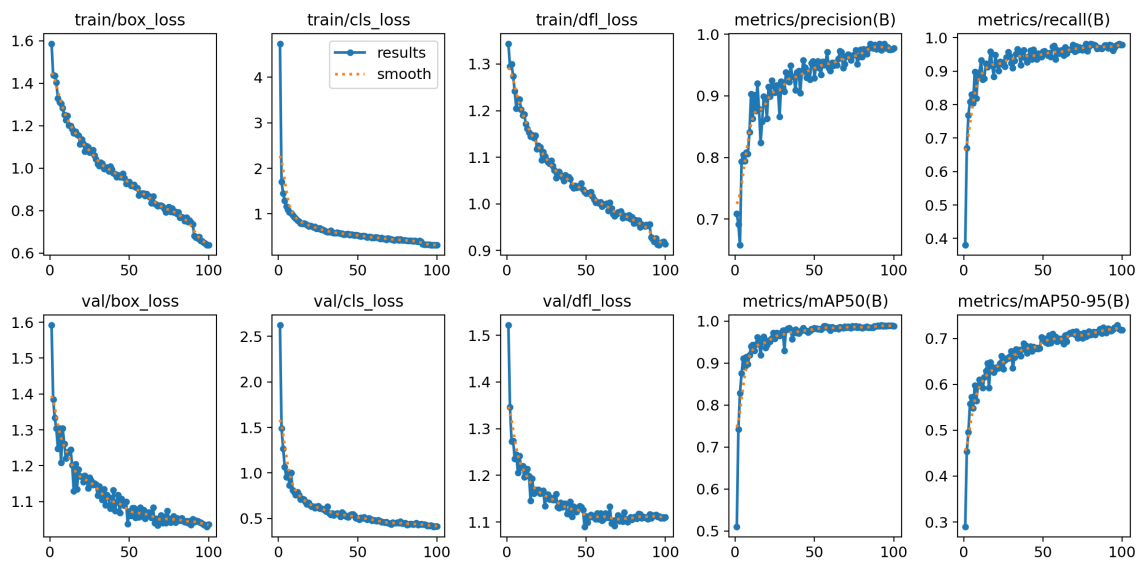
# enhance quality
close_mosaic=10,
pretrained=True,
optimizer="AdamW",
amp=True,
cache=True,

#create file
name="Genshin_Enemy_Data_Result",
exist_ok=True
)

```

4.4 Kết quả huấn luyện

Dưới đây là kết quả sau khi huấn luyện được lưu tại folder Genshin_Enemy_Data_Result :



Hình 4.1: Kết quả huấn luyện.

Các chỉ số mà chúng ta cần quan tâm đến ở đây gồm mAP50, metric/precision, metric/recall và các chỉ số loss.

- Các chỉ số loss trên biểu đồ đều giảm dần qua mỗi vòng lặp.
- mAP50 đang quá cao, điều này có thể do validation chưa đủ khó để kiểm tra mô hình.
- Precision và Recall tăng tiến đều, đặc biệt là Recall khá quan trọng trong vì thà báo nhầm còn hơn bỏ sót quái vật trong khung hình.

4.5 Trực quan hóa dữ liệu

YOLO cho phép người dùng xuất ra một folder chứa các ảnh được trực quan hóa từ tập validation bằng lệnh :

```
yolo visualize data=data.yaml
```

với data.yaml có trước từ bước huấn luyện.

Sau khi hoàn tất, trong folder chứa ảnh được trực quan hóa sẽ xuất hiện những ảnh có dạng như thế này :



Hình 4.2: Kết quả nhận diện thực tế trên tập validation.



Hình 4.3: Ảnh được trực quan hóa từ tập val với 4 phần là ground truth (đáp án thực sự), true positive (phát hiện được và phát hiện đúng), true negative (phát hiện được và phát hiện sai), false negative (hoàn toàn không phát hiện được)

4.6 Thiết lập cửa sổ quét màn hình và cảnh báo

4.6.1 Công cụ và thư viện

- YOLO
- mss, time : chụp và quét màn hình cực kì nhanh với FPS có thể đạt từ 60 - 144, cực kì phù hợp cho môi trường game.
- threading : cho phép chạy đa luồng, đa tác vụ
- pyttsx3 : thư viện giúp đọc chữ thành tiếng mà không cần tới kết nối Internet, rất nhẹ cho dự án.

4.6.2 Cấu hình của trình cảnh báo

Chúng ta sẽ setup mô hình đã huấn luyện, vùng đọc và quét màn hình, ngưỡng tin cậy và cooldown để tránh cảnh báo bị spam gây khó chịu.

- MODEL_PATH : là đường dẫn đến file best.pt (kết quả huấn luyện tốt nhất).
- GAME_REGION : 1920x1080, vì đây là kích thước cửa sổ thường thấy của người chơi Genshin Impact trên máy tính.
- CONFIDENCE_THRESHOLD : ngưỡng tin cậy, đối với dự án này nên để thấp một chút, khoảng 0.4.
- COOLDOWN : thời gian chờ cho đến thông báo mới khi phát hiện cùng một loại quái vật.

4.6.3 Thiết lập danh sách cảnh báo

Để cho cửa sổ cảnh báo có thể phát âm thanh thì trước hết chúng ta phải setup lớp quái kèm dòng chữ cảnh báo tương ứng.

```
ALERT_MESSAGES = {
    "Enemy_Cryo_Abyss_Mage": "Cryo Abyss Mage detected",
    "Enemy_Cryogunner_Legionnaire": "Cryogunner detected",
    "Enemy_Electro_Abyss_Mage": "Electro Abyss Mage detected",
    "Enemy_Fatui_Anemoboxer_Vanguard": "Fatui Anemoboxer Vanguard detected",
    "Enemy_Fatui_Cryo_Cicin_Mage": "Fatui Cryo Cicin detected",
    ...
    "Enemy_Ruin_Guard": "Ruin Guard detected",
    "Enemy_Ruin_Hunter": "ruin Hunter detected",
    "Enemy_Lawachurl": "Lawachurl detected",
}
```

Sau này, khi phát hiện quái thuộc class nào thì cửa sổ sẽ đọc dòng cảnh báo tương ứng lên thành tiếng.

4.6.4 Khởi tạo giọng nói

Hệ thống sử dụng giọng đọc mặc định, tùy vào thiết bị chạy chương trình :

```
engine = pyttsx3.init()
engine.setProperty('rate', 160)
engine.setProperty('volume', 1.0)
last_alert_time = {}
```

Các thông số rate và volume lần lượt là tốc độ đọc và âm lượng của giọng đọc; last-alert-time lưu lại thời điểm phát hiện mỗi loại quái lần cuối.

4.6.5 Phát giọng nói

Hàm cho phép cửa sổ đọc giọng nói song song mà không làm chậm FPS của chương trình chính.

```
def play_voice(text):
    def run():
        try:
            eng = pyttsx3.init()
            eng.say(text)
            eng.runAndWait()
        except:
            pass
    threading.Thread(target=run, daemon=True).start()
```

`daemon = True` đảm bảo thread tự ngắt khi cửa sổ tắt.

4.6.6 Xử lý màn hình

Đầu tiên chúng ta sẽ gọi model đã được train với cấu hình đã setup trước.

```
model = YOLO(MODEL_PATH)
```

Gọi mss để chụp màn hình liên tục

```
sct = mss.mss()
screenshot = sct.grab(GAME_REGION)
```

Ảnh mà mss trả về và ảnh mà YOLO đọc là hai dạng khác nhau, vì vậy chúng ta cần phải có công cụ chuyển đổi chúng.

```
im_np = np.array(screenshot)
img_bgr = cv2.cvtColor(im_np, cv2.COLOR_BGRA2BGR)
```

Tiếp theo YOLO sẽ dự đoán dựa trên các ảnh đã được thu thập .

```
result = model.predict(
    img_bgr,
    conf = 0.5,
    iou = 0.6,
    device = 0,
    max_det = 5,
    verbose = False,
)
```

- Confidence được đặt ở mức thấp (0.5). Khi đẩy ngưỡng này cao lên thì khả năng báo sai sẽ giảm nhưng khả năng bỏ sót có thể tăng lên. Sau khi suy xét yêu cầu của trình cảnh báo, 0.5 là một ngưỡng chấp nhận được.
- `device = 0` : dĩ nhiên rồi, chúng ta sẽ ép nó dùng GPU thay vì CPU vì nó có tốc độ xử lý nhanh hơn rất nhiều so với CPU.
- `max_det = 5` : mặc dù các quái vật lớn xuất hiện đơn lẻ và ít theo bầy đàn, vì vậy không nên để nó cao.

Hiển thị cửa sổ để biết được nó có đang hoạt động và nhìn vào màn hình hay không, đồng thời có thể tắt cửa sổ chỉ bằng một phím bấm.

```
preview_img = cv2.resize(img_bgr, (640, 360))
cv2.imshow('Press Q to exit', preview_img)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break
```

4.6.7 Đánh giá sơ bộ về cửa sổ cảnh báo

Sau khi chạy thử với video gameplay và chạy thực thể trong game, trình cảnh báo có một số đặc điểm như sau :

- Phát hiện và cảnh báo được kẻ địch ở mức trung bình, cụ thể :
 - Đối với các quái lớn như các phiên bản của Máy di tích, tỉ lệ phát hiện và cảnh báo chính xác hầu như là tuyệt đối dưới nhiều vị trí và góc độ.
 - Các quái có model giống nhau và model nhỏ như Fatui Bắn tĩa và Fatui Nham có xảy ra nhầm lẫn hoặc không thể phát hiện, điều tương tự cũng thi thoảng xảy ra ở các quái vật cùng loài nhưng khác màu như Pháp sư Vực sâu Hỏa và Băng.
 - Hay mắc lỗi phát hiện ra được và phát hiện đúng loại quái vật nhưng không phát cảnh báo.
- Gần như không ảnh hưởng đến FPS khi chơi game, trải nghiệm mượt mà, không bị giật lag.
- Âm thanh cảnh báo dễ nghe, rõ ràng.

Chương 5

Kết luận và hướng phát triển

5.1 Kết luận

Sau khi thực hiện nghiên cứu, thu thập dữ liệu, huấn luyện và tạo trình cảnh báo với YOLOv8, đội ngũ thực hiện dự án có thể đưa ra những kết luận chính như sau:

- **Tốc độ và hiệu năng ở mức khá :** YOLOv8 đảm bảo cho thời gian suy luận và phát hiện vật thể khá nhanh, phù hợp với mục tiêu dự án.
- **Độ chính xác trong huấn luyện và độ chính xác trong thực tế :** độ chính xác sau huấn luyện rất cao với yolov8s. Tuy nhiên khi mang vào môi trường game thực tế, hiệu năng của trình cảnh báo chỉ đạt mức trung bình.
- **An toàn cho tài khoản của người chơi :** mã nguồn và kiểm thử thực tế trên các tài khoản khác nhau cho thấy trình cảnh báo không can thiệp vào mã nguồn của game cũng như vi phạm chính sách về phần mềm gian lận.

5.2 Hướng phát triển:

- Nâng cấp mô hình lên YOLOv8m hoặc cao hơn để cải thiện khả năng nhận diện những quái vật nhỏ.
- Mở rộng dữ liệu vì dữ liệu cho dự án hiện đang khá ít, đặc biệt là dữ liệu cho các điều kiện môi trường khắc nghiệt khác như bão cát, tuyết rơi dày, ...
- Xây dựng giao diện người dùng (GUI) để người chơi dễ dàng thao tác.
- Tích hợp nhiều voice hơn để nó quen thuộc với người chơi Genshin Impact

Những hướng phát triển này không chỉ nâng cao hiệu quả mô hình trong bài toán phát hiện và nhận diện kẻ địch mà còn khiến trình cảnh báo có thể tùy biến hơn để phù hợp với cộng đồng người chơi Genshin Impact.

Tài liệu tham khảo

- [1] Glenn Jocher, et al., “Ultralytics YOLOv8,” *Ultralytics*, 2023. [Online], <https://github.com/ultralytics/ultralytics>.
- [2] Z. Zou, K. Chen, Z. Shi, Y. Guo, và J. Ye, “Object Detection in 20 Years: A Survey,” *Proceedings of the IEEE*, tập 111, số 3, trang 257–276, 2023.
- [3] Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression, <https://arxiv.org/abs/1911.08287>